CSE 554 Machine Learning Systems

Lecture 8 - Quantization

Baris Kasikci

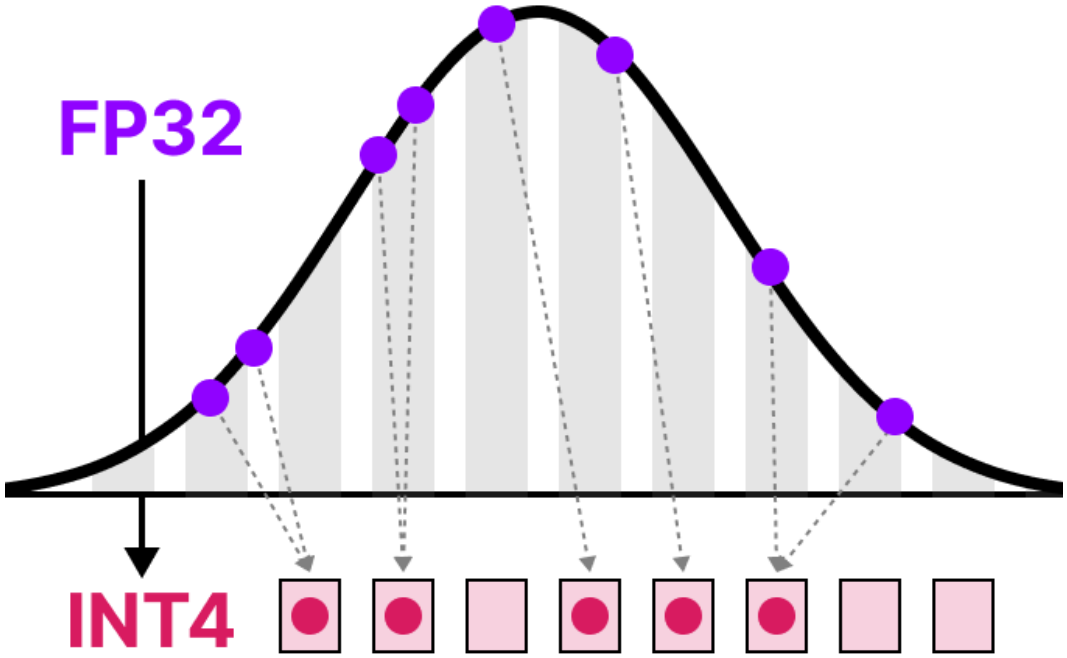
BE BOUNDLESS

Class Overview

1. GPU Architecture and Programming
2. Transformers (Architecture and Implementation)
3. Performance modeling
4. **Quantization**
5. Memory Management
6. Sparsity
7. Mixture of Experts (MoE)
8. Batching
9. Speculative decoding
10. Parallelism
11. End-to-end serving
12. Agentic systems
13. Large-scale deployment

Quantization

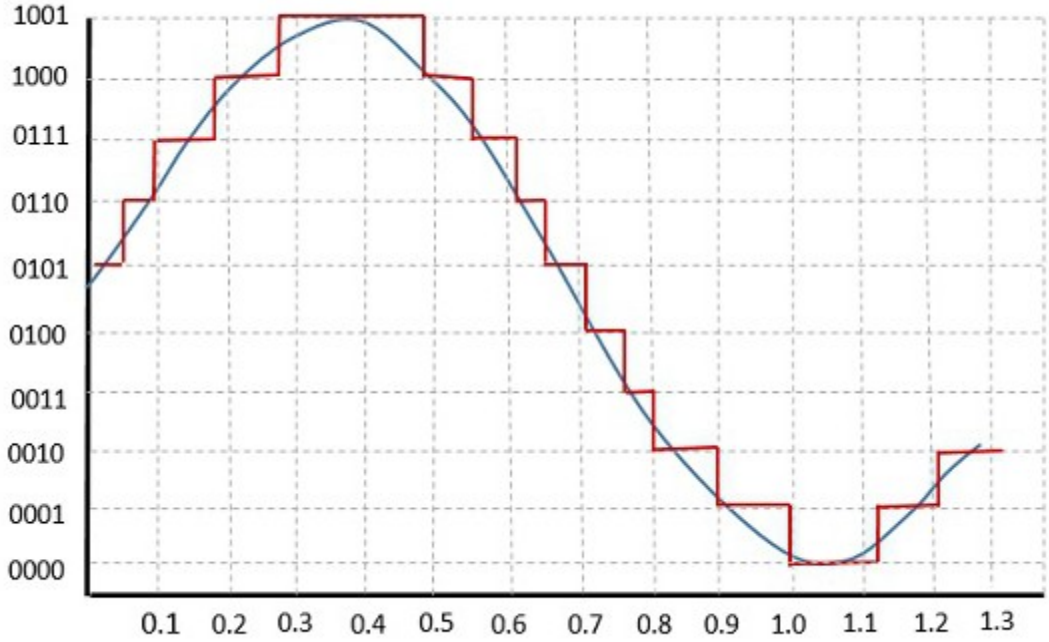
Key Idea



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-

Reduced memory footprint, latency, and energy via a compact representation

Quantization Creates a Step Function

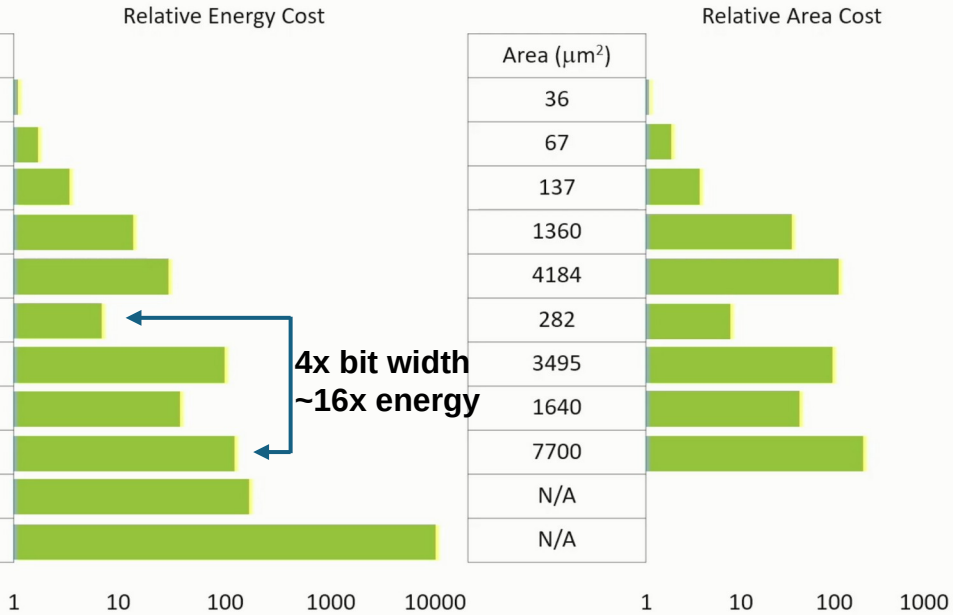


https://www.tutorialspoint.com/digital_communication/digital_communication_quantization.htm

Motivation Behind Quantization – Energy

Cost of Operations

Operation:	Energy (pJ)	Area (μm²)
8b Add	0.03	36
16b Add	0.05	67
32b Add	0.1	137
16b FP Add	0.4	1360
32b FP Add	0.9	4184
8b Mult	0.2	282
32b Mult	3.1	3495
16b FP Mult	1.1	1640
32b FP Mult	3.7	7700
32b SRAM Read (8KB)	5	N/A
32b DRAM Read	640	N/A



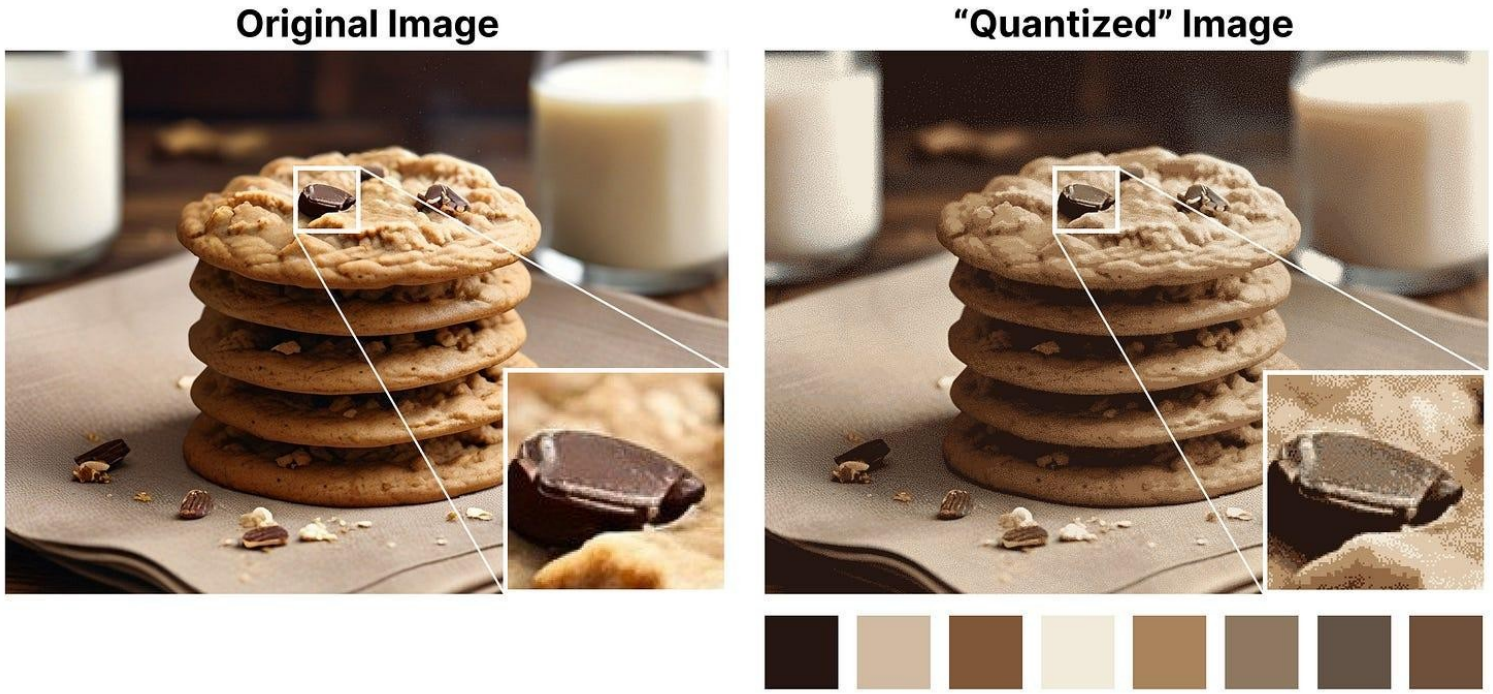
Energy consumption of multiplication grows quadratically with bit-width

Motivation wrt LLMs

- LLMs were initially primarily memory-bound
- Lots of optimizations went into addressing this (we covered/will cover all of the below)
  - Quantization
  - Sparsity optimizations
  - KV Cache
  - FlashAttention
  - PagedAttention
  - GQA

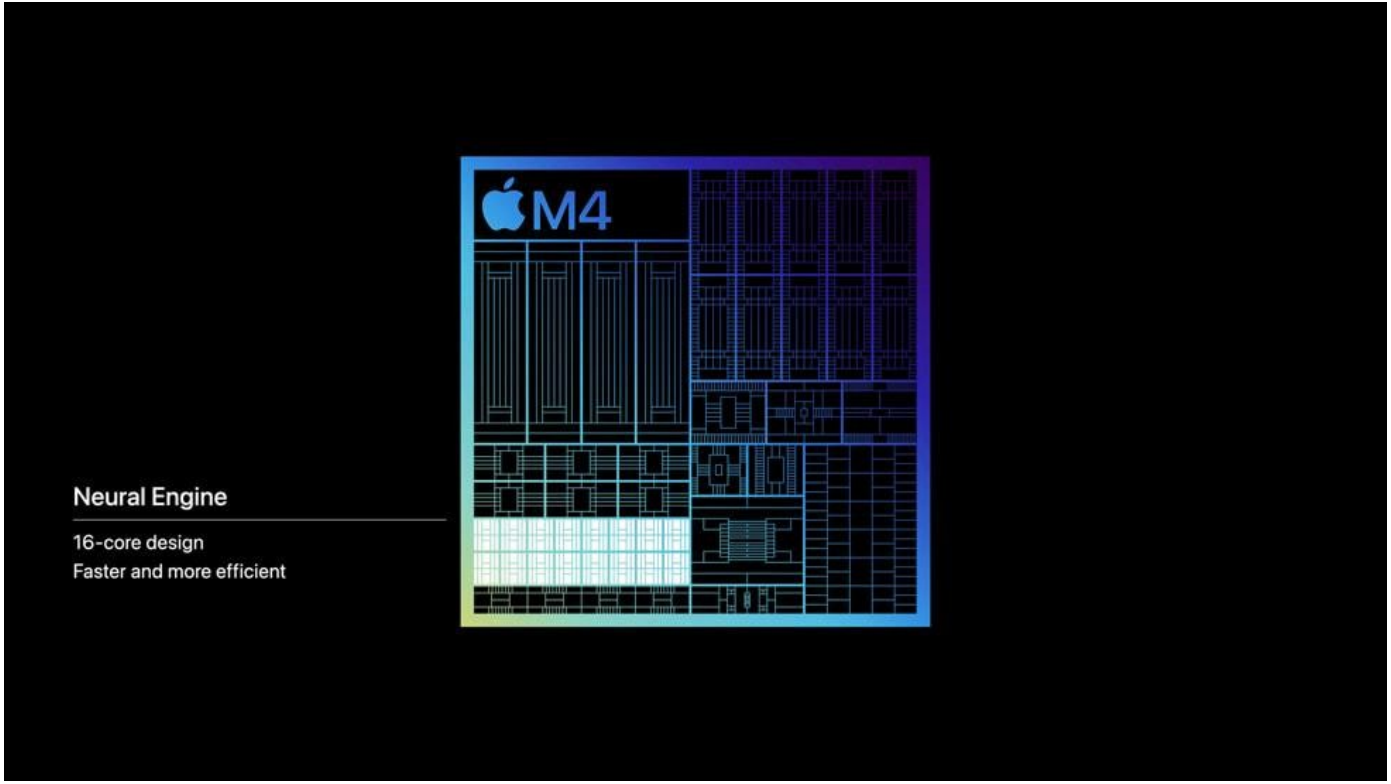
These improvements have pushed transformers to a compute-bound regime

Quantization Introduces Precision Loss



https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

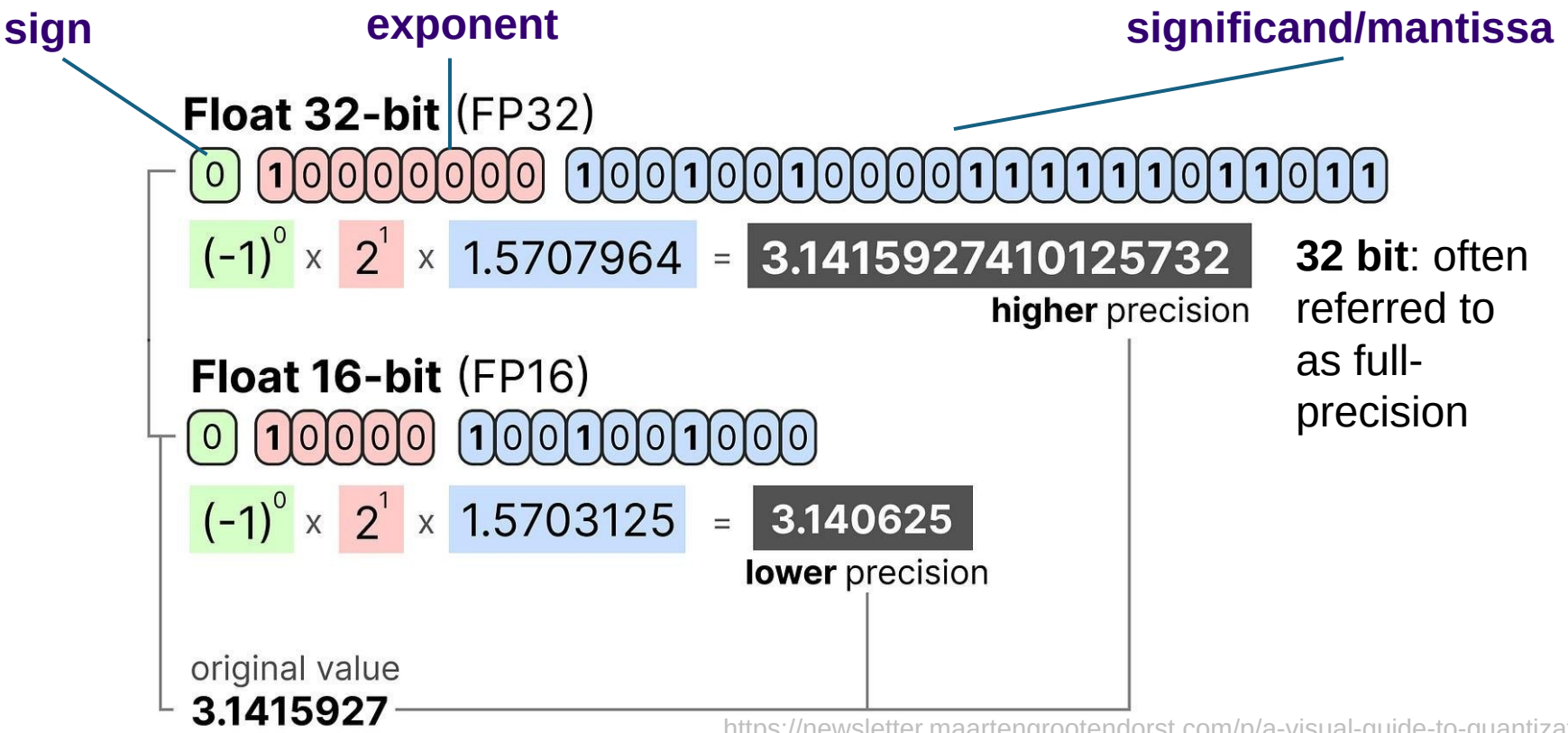
Quantization in Real Systems



Apple Neural Engine leverages quantization

https://www.apple.com/newsroom/2024/05/apple-introduces-m4-chip/

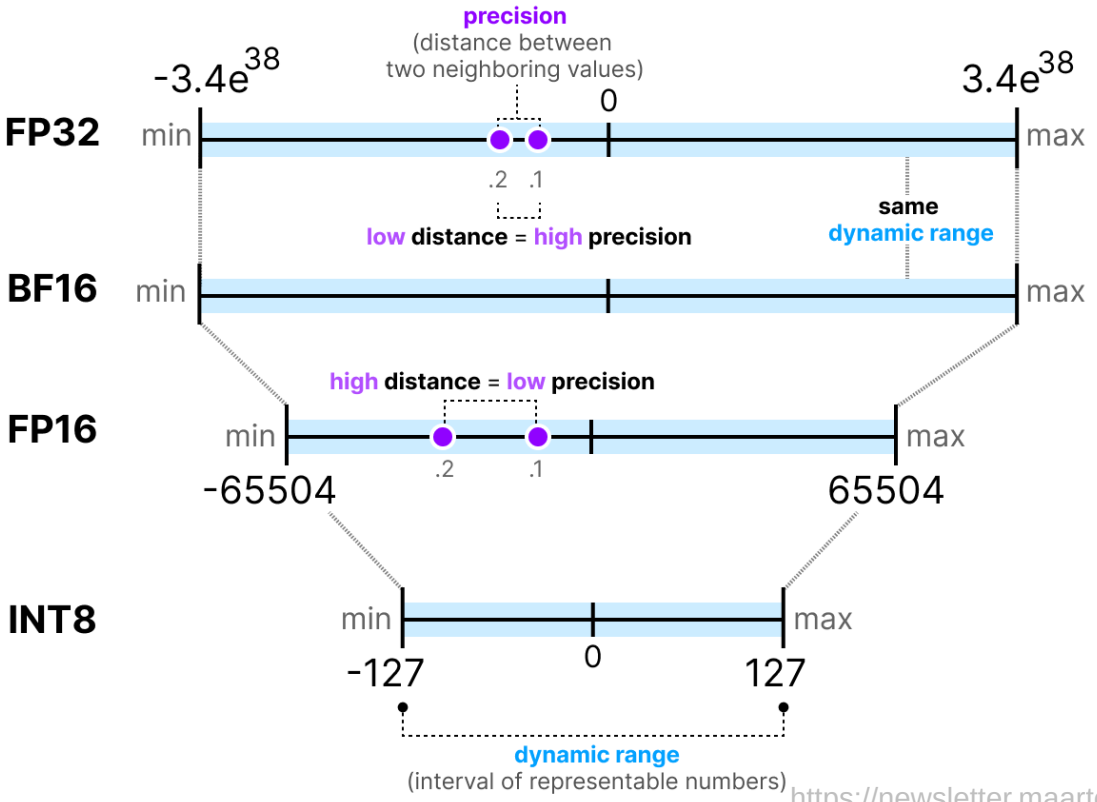
Floating Point Representation



32 bit: often referred to as full-precision

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

Dynamic Range and Precision

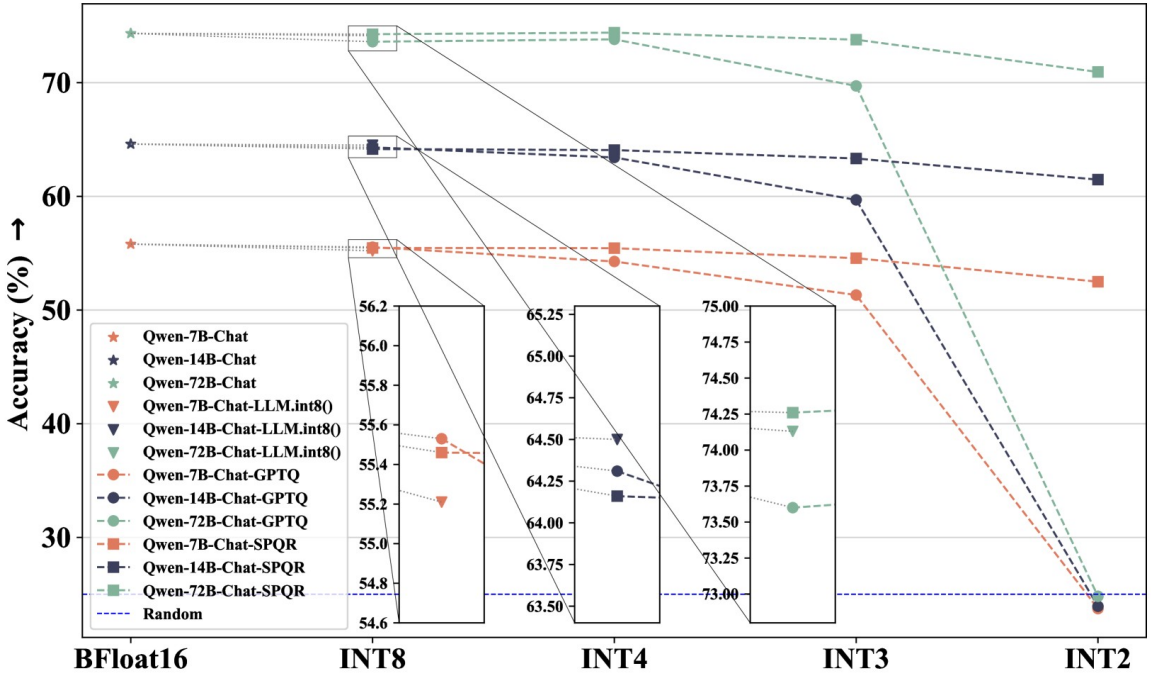


Dynamic range: Range of representable numbers (higher range is better for training)

Precision: Distance between two neighboring values (higher precision is better for inference)

https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization

Quantization and Accuracy

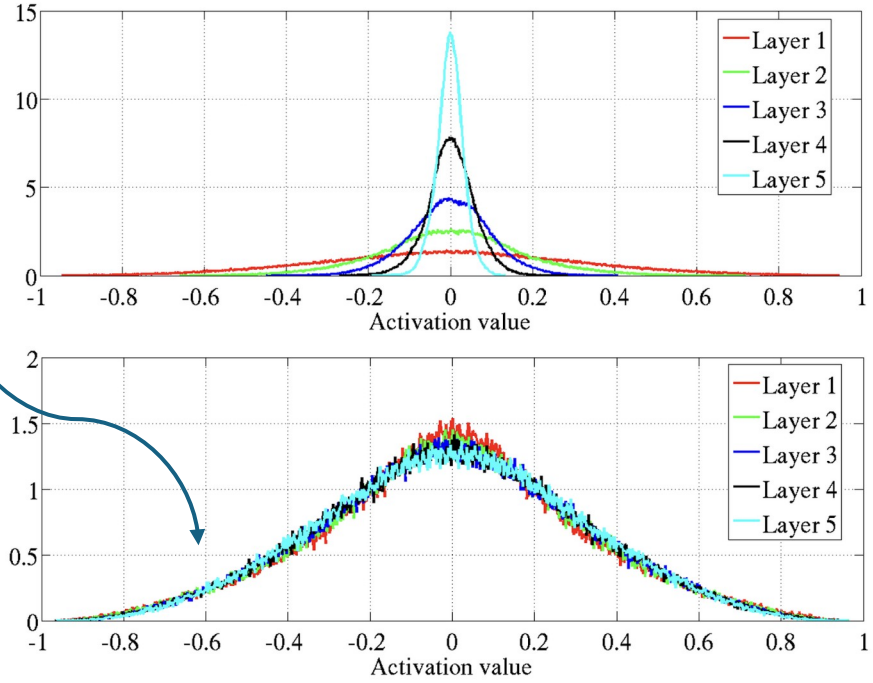


https://arxiv.org/abs/2402.16775v2

High variance in how accuracy decreases with more aggressive quantization

Why Does Quantization Work?

- Activation (and parameter) ranges are well-defined
  - They are distributed evenly
  - Initializing activations to have even distributions leads to better training outcomes [Glorot & Bengio, 2010]
- Activations diverge without proper initialization
  - Activations diverge, leading to unstable gradients



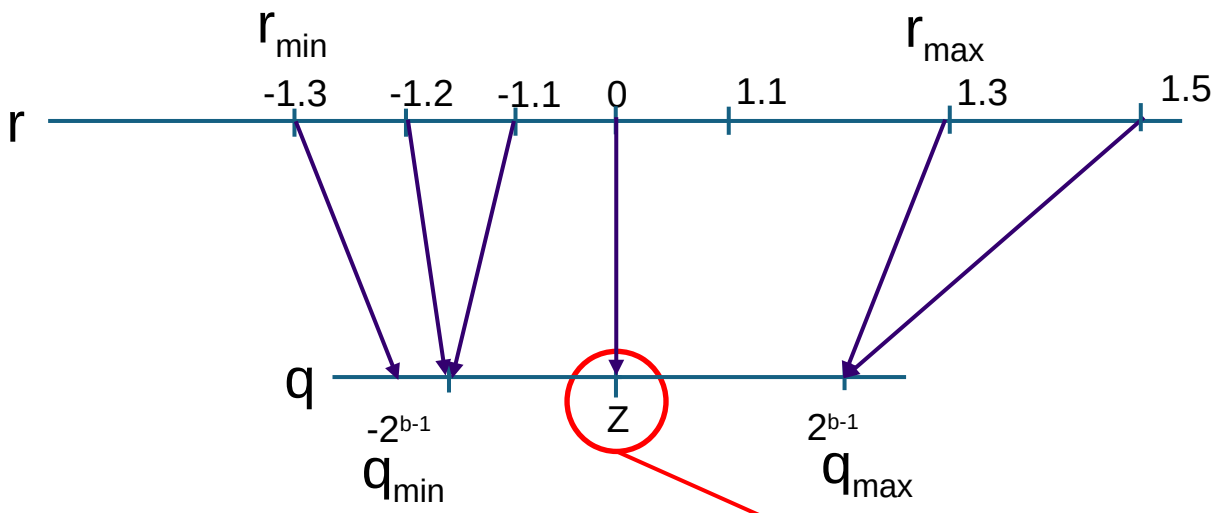
Quantization and Modern Models

- Deepseek V3
  - With FP8 quantization (what Deepseek primarily uses)
    - 671B parameters, 1 Byte each = 671 GB
      - Can fit in ~5 H200s with HBM (high-bandwidth memory)
  - With BF16 model weights (supported via a conversion script)
    - 671B parameters, 2 Bytes each = 1,342 GB (1.3 TB)

Training/Inference Workflow & Quantization

- Post-training quantization (PTQ)
  - Training uses non-quantized (precise and larger) data types
  - Inference uses quantization (of weights and/or activations)
- Quantization aware training (QAT)
  - Training simulates quantization, making the NN robust to quantization error
- Low-precision training with quantized data
  - E.g., Binary neural networks [Rastegari et al., 2016]

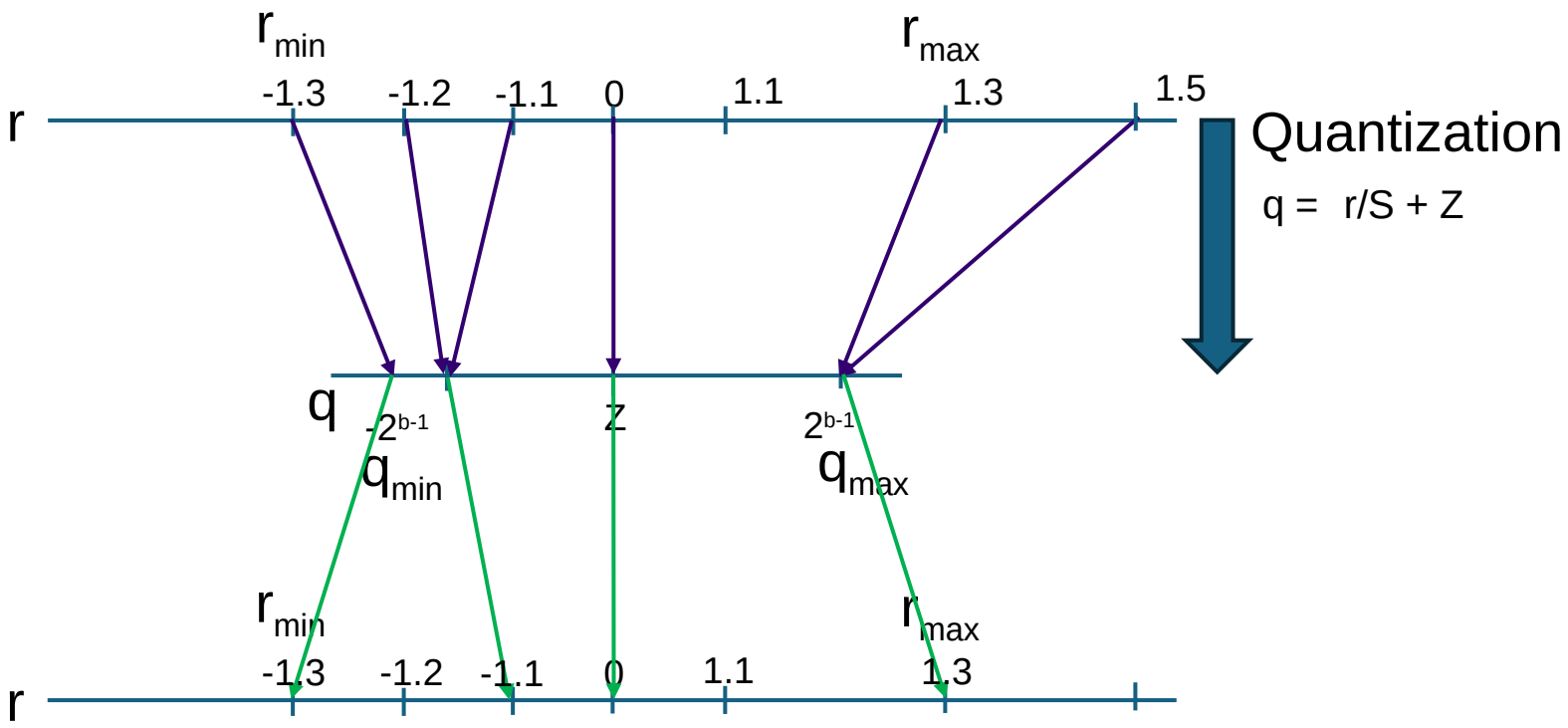
Linear Quantization



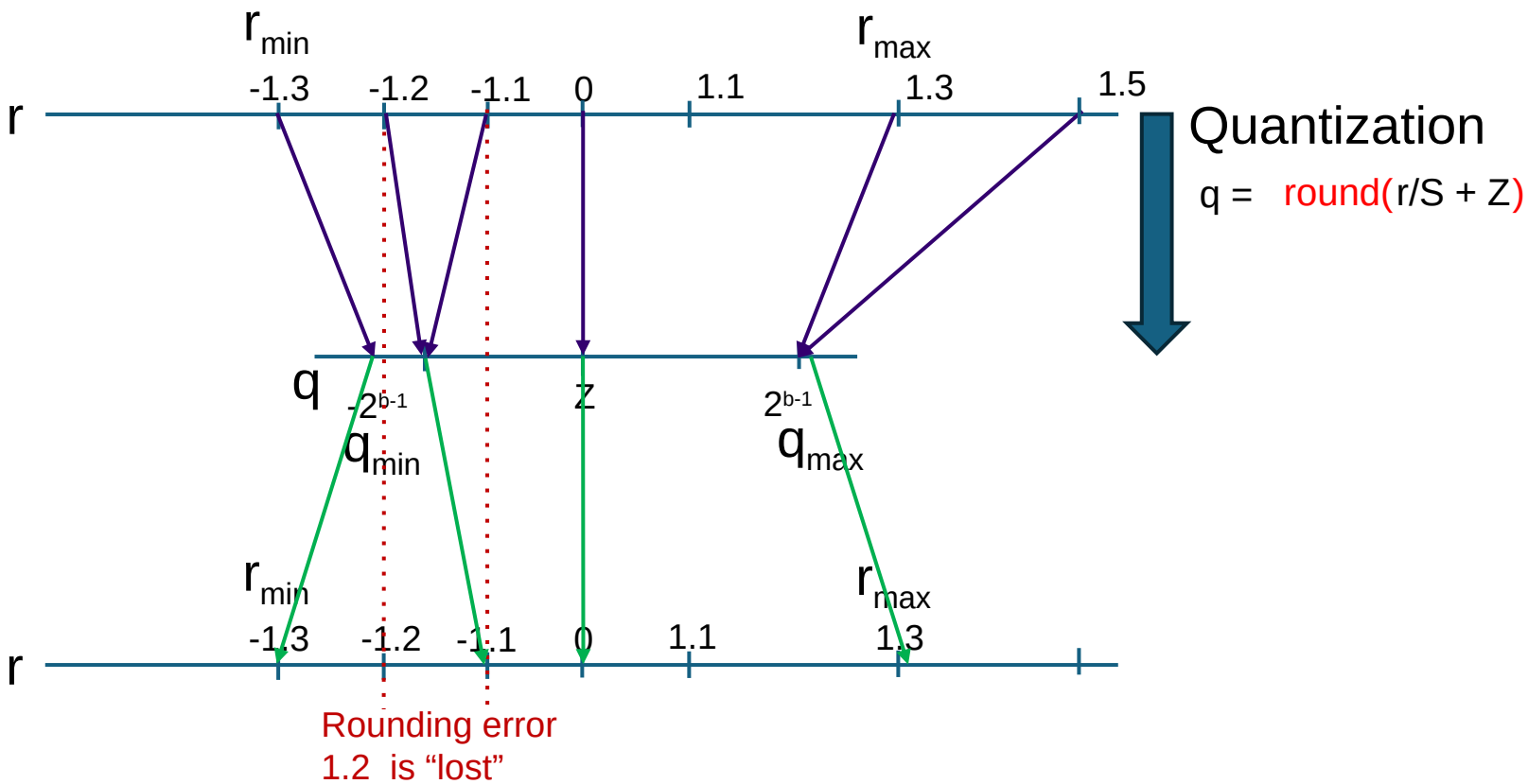
If the zero value is 0, this is called
symmetric quantization

Adapted from https://abdelfattah-class.github.io/ece5545/

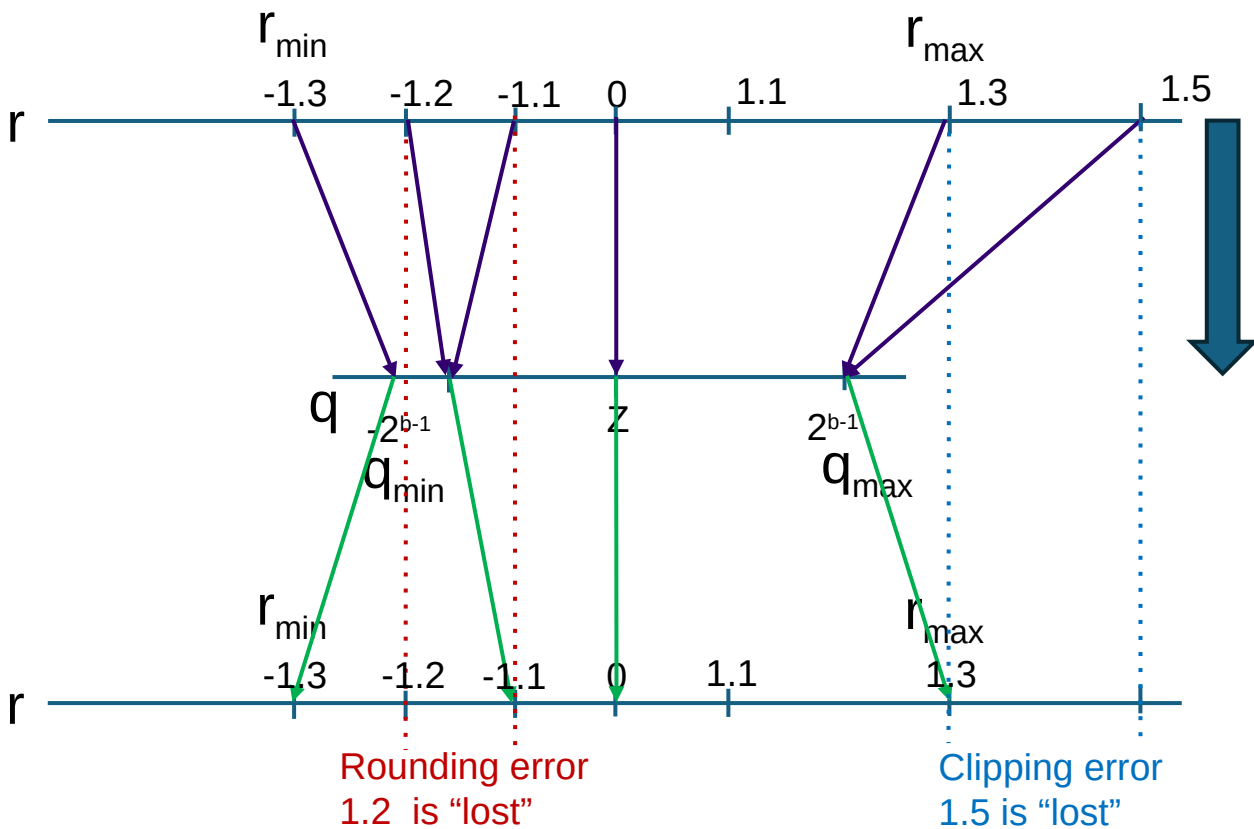
Linear Quantization



Linear Quantization



Linear Quantization

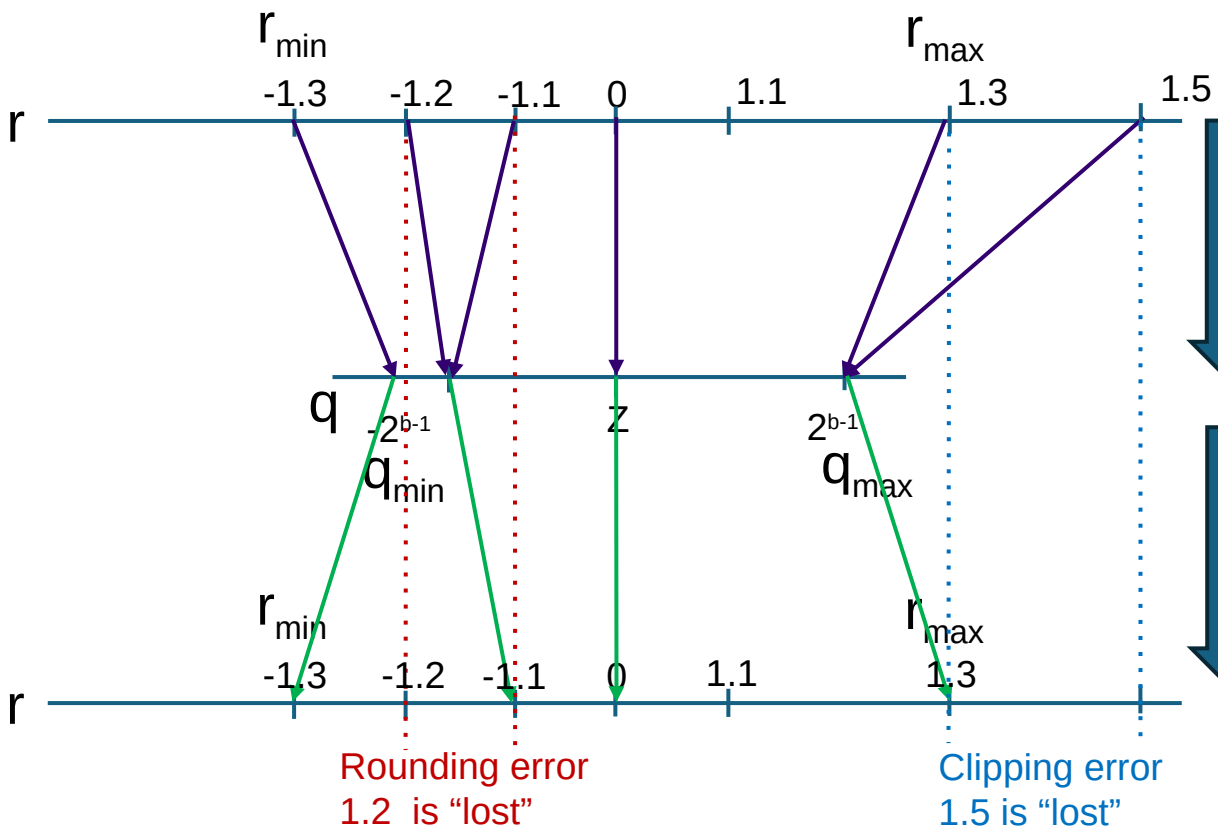


Quantization

$$q = \text{clip}(\text{round}(r/S + Z), -2^{b-1}, 2^{b-1})$$

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

Linear Quantization



Quantization

$$q = \text{clip}(\text{round}(r/S + Z), -2^{b-1}, 2^{b-1})$$

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

Dequantization

$$r = S(q - Z)$$

Rounding Error

- Rounding to nearest is bounded [-S/2, S/2]
- Can be reduced by reducing r_{max} and increasing r_{min}
  - Trade-off: clipping error will increase
- Rounding schemes impact accuracy a lot

Rounding scheme	Acc(%)
Nearest	52.29
Ceil	0.10
Floor	0.10
Stochastic	52.06±5.52
Stochastic (best)	63.06

Table 1. Comparison of ImageNet validation accuracy among different rounding schemes for 4-bit quantization of the first layer of Resnet18. We report the mean and the standard deviation of 100 stochastic (Gupta et al., 2015) rounding choices (Stochastic) as well as the best validation performance among these samples (Stochastic (best)).

Quantization Example

- [Quantization example in Colab]

```python
import torch

a = torch.randn(5)
# a[-1] = -5 # Manually add an outlier
print(a)

# find normalization constant
amax = torch.abs(a).max()

# scale it into range [-127, 127]
scaled = a/amax*127
# print(scaled)

# round to the nearest value
int8val = torch.round(scaled)
# print(int8val)

# dequantization
fp32val = (int8val.float()*amax/127.0)

print(f"Original {a}")
print(f"Quantized: {fp32val}")
print(f"Average error: {torch.abs(fp32val - a).mean()}")
```

```
tensor([ 0.1673,  0.5204,  0.8255, -0.3330,  0.4819])
Original tensor([ 0.1673,  0.5204,  0.8255, -0.3330,  0.4819])
Quantized: tensor([ 0.1690,  0.5200,  0.8255, -0.3315,  0.4810])
Average error: 0.0008921354892663658
```

https://www.youtube.com/watch?v=o94ODz1CAtk

Linear Quantization and Matrix Multiplication

$$Y = WX$$

Quantization

$$Y = S_w(q_w - Z_w)S_x(q_x - Z_x)$$

$$r = S(q - Z)$$

$$Y = S_w\, S_x\, [q_w q_x - q_w Z_x - q_x Z_w + Z_w\, Z_x]$$

From https://efficientml.ai

Linear Quantization and Matrix Multiplication

$$Y = WX$$

Quantization

$$Y = S_w(q_w - Z_w)S_x(q_x - Z_x)$$

$$r = S(q - Z)$$

$$Y = S_w S_x q_w q_x - S_w S_x q_w Z_x - S_w S_x Z_w q_x + S_w S_x Z_w Z_x$$

Linear Quantization and Matrix Multiplication

$$Y = WX$$

Quantization

$$Y = S_w(q_w - Z_w)S_x(q_x - Z_x)$$

$$r = S(q - Z)$$

$$Y = \underbrace{S_w S_x q_w q_x}_{\text{Symmetric Quantization}} - S_w S_x q_w Z_x - \underbrace{S_w S_x Z_w q_x}_{\text{Online overhead due to Asymmetric Quantization}} + S_w S_x Z_w Z_x$$

Symmetric Quantization
$Z_w = Z_x = 0$

Online overhead due to Asymmetric Quantization

Grayed-out values can be determined offline. Weights are known after training, for activations, use calibration/profiling data

Linear Quantization and Matrix Multiplication

$$Y = WX$$

Quantization

$$Y = S_w(q_w - Z_w)S_x(q_x - Z_x)$$

$$r = S(q - Z)$$

$$Y = \underbrace{S_w S_x q_w q_x}_{\text{Symmetric Quantization}} - S_w S_x q_w Z_x - \underbrace{S_w S_x Z_w q_x}_{\text{Online overhead due to Asymmetric Quantization}} + S_w S_x Z_w Z_x$$

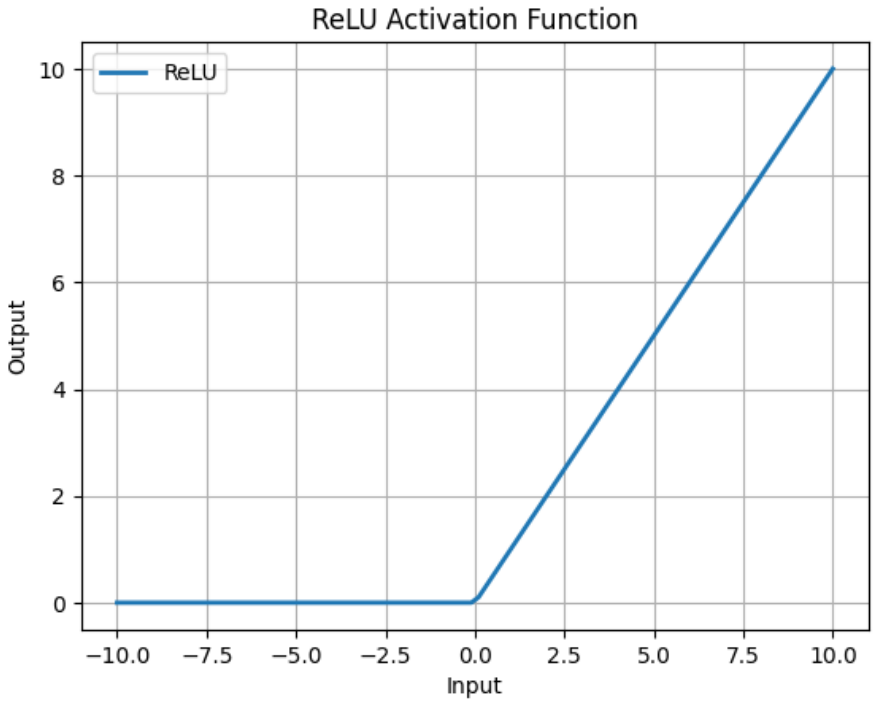
Symmetric Quantization
$Z_w = Z_x = 0$

Online overhead due to Asymmetric Quantization

Can eliminate using symmetric quantization for weights, $Z_w = 0$

Why Would You Ever Use Asymmetric Quantization?

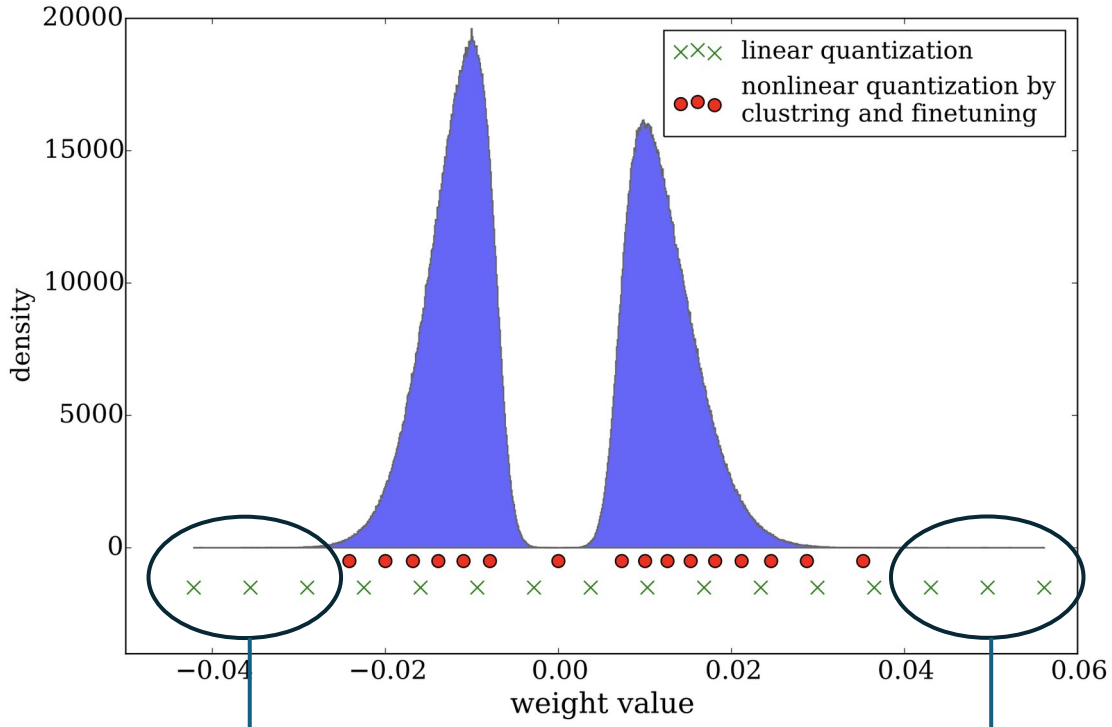
- Asymmetric quantization is more flexible
  - E.g., in ReLU almost all activations are positive, so a scheme like asymmetric quantization is more useful to apply to the output of ReLU



https://colab.research.google.com/drive/1qXNi6LIOWPAOX_Tjt7UDPKN1_ZpUCkXp?usp=sharing

Non-Linear Quantization – What's the Need?

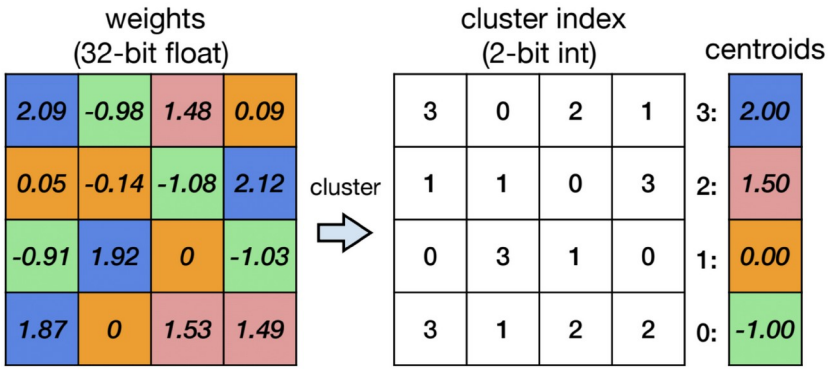
- Useful for skewed distributions



Linear quantization reduces precision

https://arxiv.org/pdf/1510.00149

Non-Linear Quantization - Clustering



Determined using clustering

https://efficientml.ai, Google ML course

Non-Linear Quantization - Clustering

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

cluster ⇨

cluster index
(2-bit int)

3	0	2	1
1	1	0	3
0	3	1	0
3	1	2	2

centroids

3:	2.00
2:	1.50
1:	0.00
0:	-1.00

Dequantization
refers
to the codebook

reconstructed weights
(32-bit float)

2.00	-1.00	1.50	0.00
0.00	0.00	-1.00	2.00
-1.00	2.00	0.00	-1.00
2.00	0.00	1.50	1.50

quantization error

0.09	0.02	-0.02	0.09
0.05	-0.14	-0.08	0.12
0.09	-0.08	0	-0.03
-0.13	0	0.03	-0.01

		indexes		codebook		
storage	32 bit **x** 16 64B	2 bit **x** 16 32 bit = 4B	+	32 bit **x** 4 128 bit = 16B	=	20B

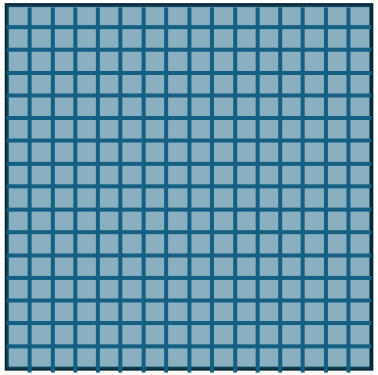
3.2x smaller

N centroids → $\log_2$N indices << Number of parameters

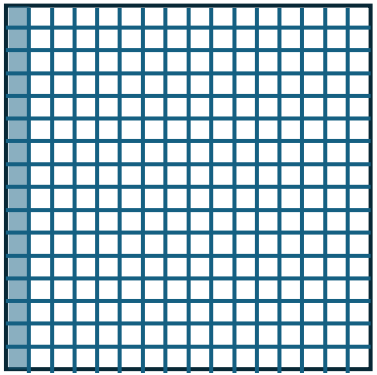
Non-linear quantization is primarily only applicable to weights

Granularity of Quantization

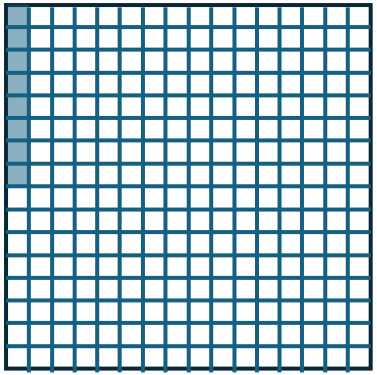
Per-Tensor



Per-Channel/Vector*



Per-Group



Benefits of finer-grained quantization

- More precise

Drawbacks of finer-grained quantization

- More complex/higher overhead

[*] https://arxiv.org/pdf/2102.04503

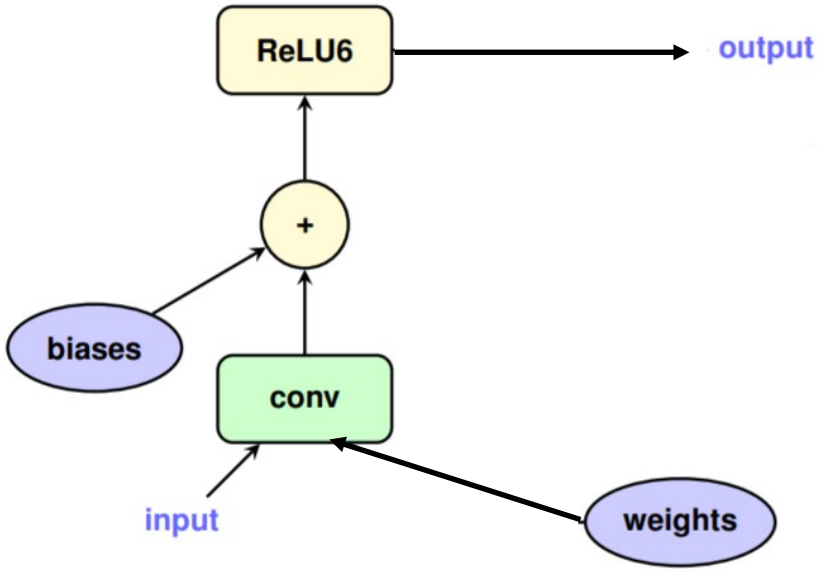
Training/Inference Workflow & Quantization

- Post-training quantization (PTQ)
  - Training uses non-quantized (precise and larger) data types
  - Inference uses quantization (of weights and/or activations)
- Quantization aware training (QAT)
  - Training simulates quantization, making the NN robust to quantization error
- Low-precision training with quantized data
  - E.g., Binary neural networks [Rastegari et al., 2016]

Training/Inference Workflow & Quantization

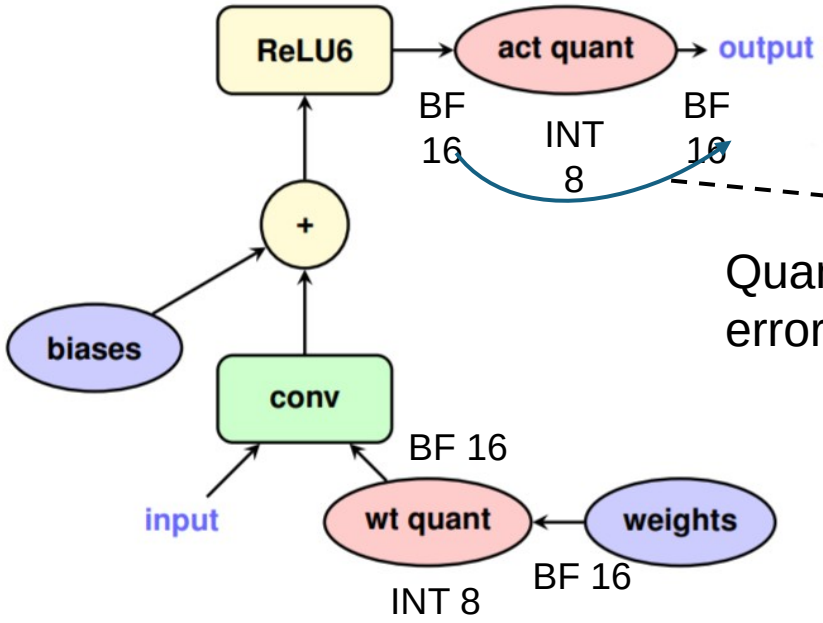
- Post-training quantization (PTQ)
  - Training uses non-quantized (precise and larger) data types
  - Inference uses quantization (of weights and/or activations)
- Quantization aware training (QAT)
  - Training simulates quantization, making the NN robust to quantization error
- Low-precision training with quantized data
  - E.g., Binary neural networks [Rastegari et al., 2016]

Quantization-Aware Training (QAT)



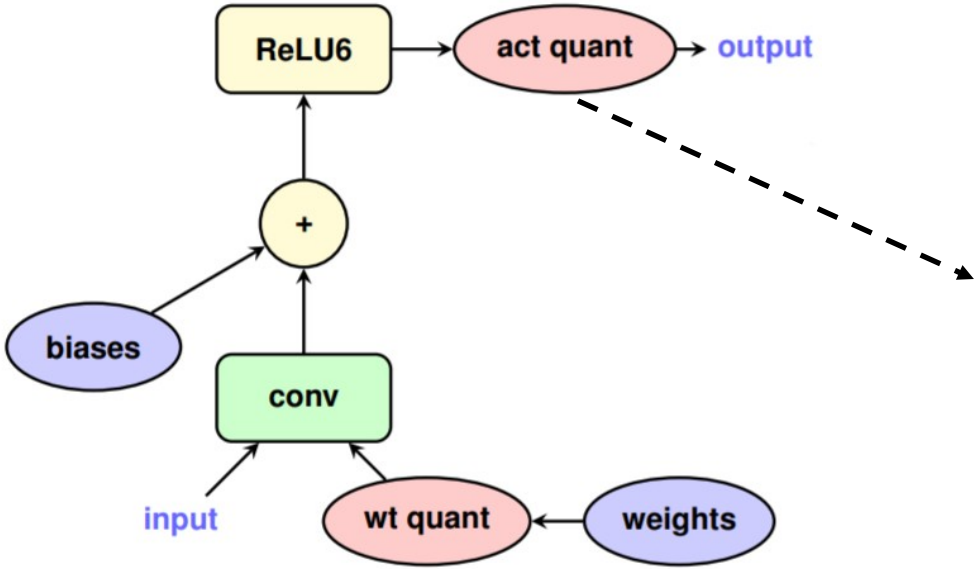
https://blog.tensorflow.org/

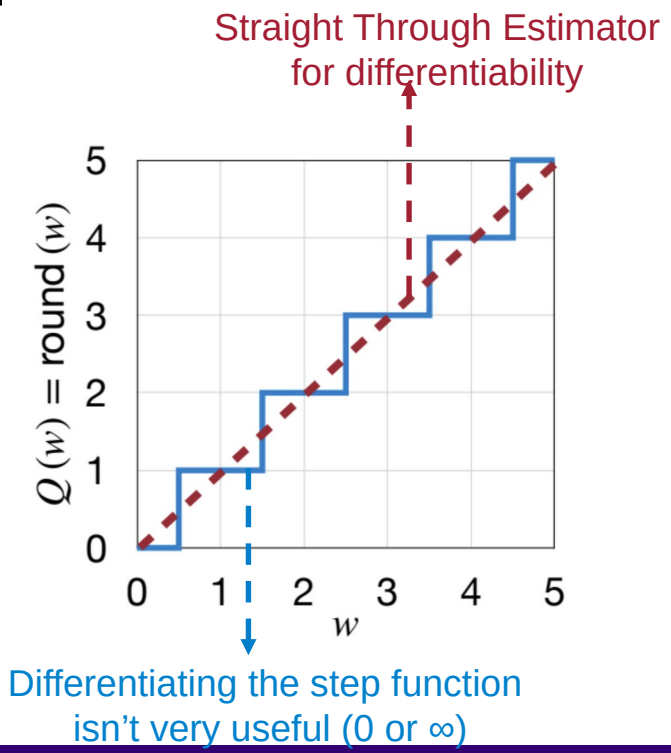
Quantization-Aware Training (QAT)



Quantization/dequantization produces the error that would occur during inference

QAT and Backpropagation





Straight through Estimator (STE) makes quantization operators differentiable for backpropagation

How Well Does QAT Do?

Model	Floating-point baseline model	QAT model	Delta	Post-training full integer quantized model
MobileNet v1 1.0 224	71.03%	71.06%	0.04%	69.57%
MobileNet v2 1.0 224	70.77%	70.01%	-1.07%	70.2%
ResNet v1 50	76.3%	76.1%	-0.26%	75.95%

QAT mostly outperforms PTQ

Quantization in the Era of LLMs

LLM.int8()

SmoothQuant

AWQ

LLM.int8()

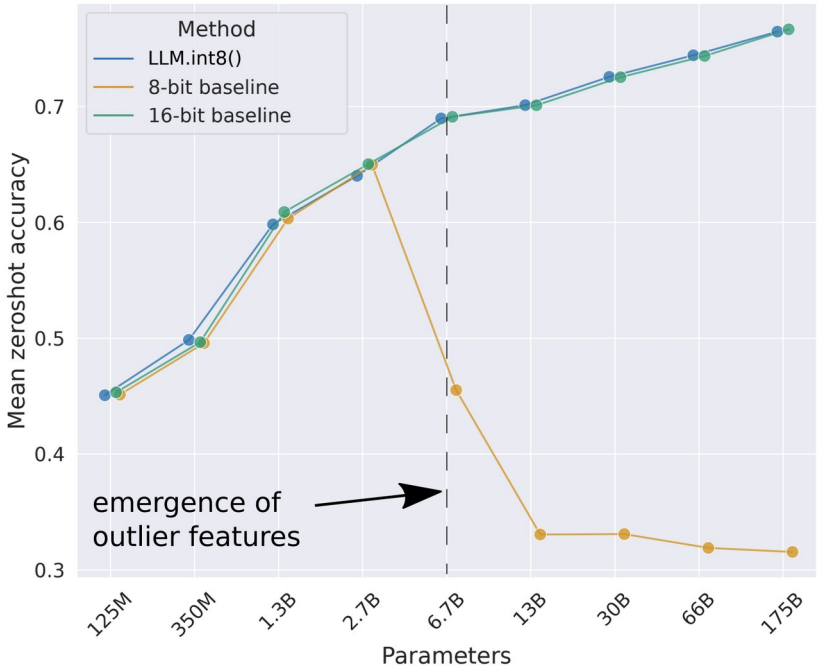


Figure 1: OPT model mean zeroshot accuracy for WinoGrande, HellaSwag, PIQA, and LAMBADA datasets.

Outliers in LLMs cause sharp accuracy degradation when quantizing

Outliers

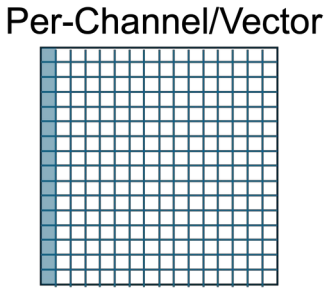
- Quantization example in Colab

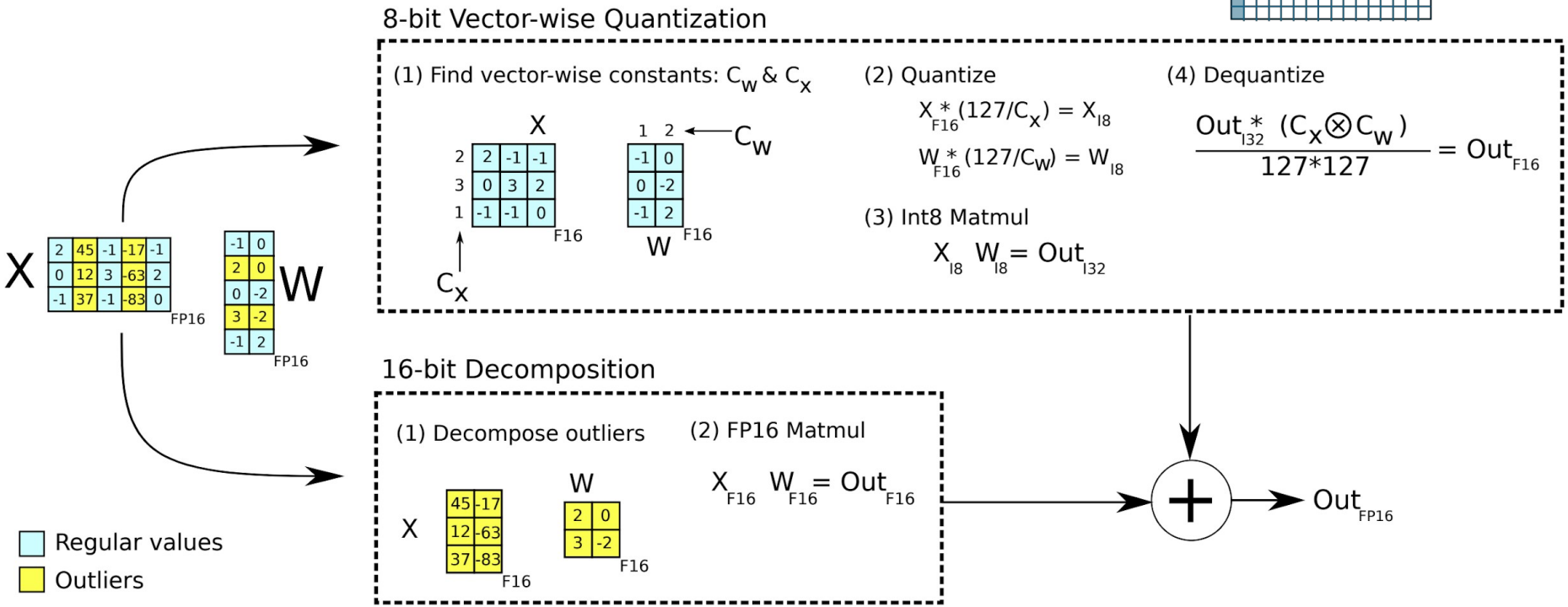
```python
import torch

a = torch.randn(5)
a[-1] = -5 # Manually add an outlier
print(a)

# find normalization constant
amax = torch.abs(a).max()

# scale it into range [-127, 127]
scaled = a/amax*127
# print(scaled)

# round to the nearest value
int8val = torch.round(scaled)
# print(int8val)

# dequantization
fp32val = (int8val.float()*amax/127.0)

print(f"Original {a}")
print(f"Quantized: {fp32val}")
print(f"Average error: {torch.abs(fp32val - a).mean()}")
```

```
tensor([-0.4837,  0.2219,  0.9545,  0.1632, -5.0000])
Original tensor([-0.4837,  0.2219,  0.9545,  0.1632, -5.0000])
Quantized: tensor([-0.4724,  0.2362,  0.9449,  0.1575, -5.0000])
Average error: 0.008188093081116676
```

https://www.youtube.com/watch?v=o94ODz1CAtk

LLM.int8()Idea 1: Vector-Wise Quantization





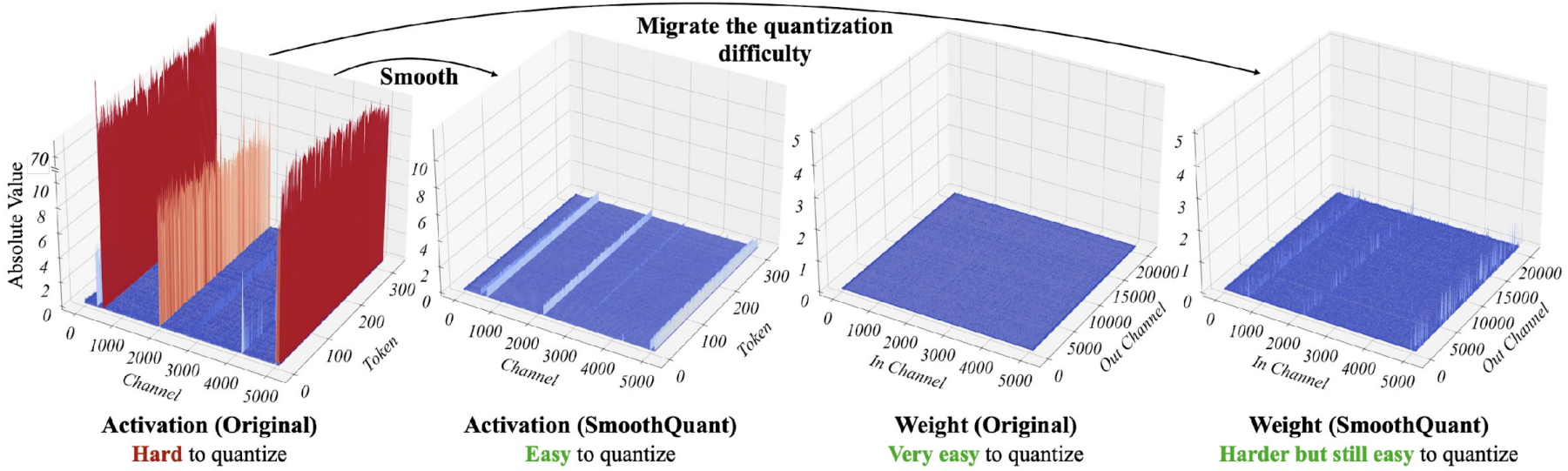
https://arxiv.org/pdf/2208.07339

SmoothQuant – W8A8 Quantization

- Same motivation
  - Classical quantization doesn’t work with LLMs due to outliers
- Observation
  - In LLMs outliers are typically in activations, not weights!
- Key idea: migrate the quantization difficulty
  - Multiply weights and suppress activations to make them more smooth
  - Weights will be harder to quantize but not that hard
  - Use 8 bits for weights and activations

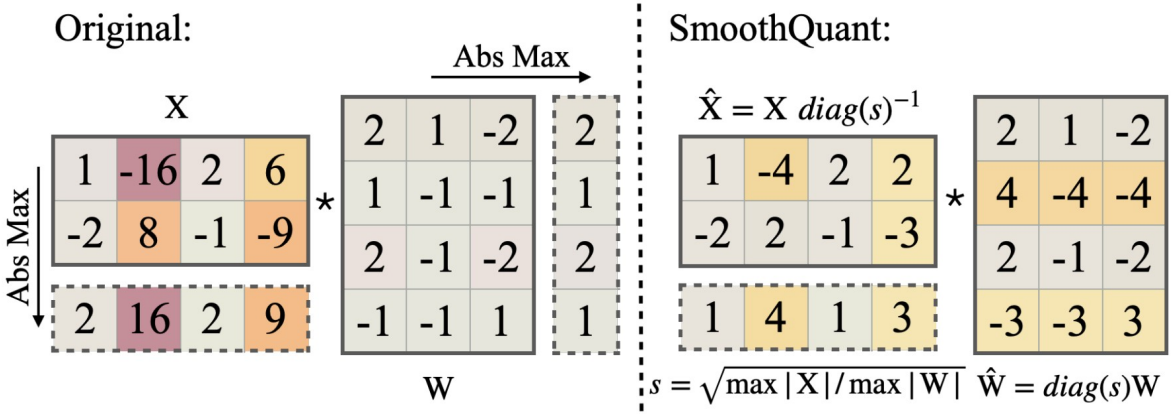
https://www.youtube.com/watch?v=U0yvqjdMfr0

SmoothQuant



SmoothQuant -Computing a Scaling Factor

$$\mathbf{s}_j = \max(|\mathbf{X}_j|)^{\alpha} / \max(|\mathbf{W}_j|)^{1-\alpha}$$



Authors empirically find α=0.5 to provide a “well-balanced split”

Balancing the quantization difficulty between weights and activations is helpful

AWQ – Activation-Aware Weight-Only Quantization

- SmoothQuant is suitable for batched scenarios
- For low-batch sizes or embedded systems, W8 is prohibitive
- Weights take up more space compared to activations
  - Recall the transformer implementation:
    - hidden_dim x (num_heads*head_dim) >> seq_len x hidden_dim
- AWQ focuses on low-bit weight-only quantization

https://www.youtube.com/watch?v=3dYLj9vjfA0

W4 Quantization

$\mathbf{W}_{\text{FP16}}$

+1.2	−0.2	−2.4	−3.4
−2.5	−3.5	+1.9	+1.4
−0.9	+1.6	−2.5	−1.9
−3.5	+1.5	+0.5	−0.1
+1.8	−1.6	−3.2	−3.4
+2.4	−3.5	−2.8	−3.9
+0.1	−3.8	+2.4	+3.4
+0.9	+3.3	−1.9	−2.3

$\xrightarrow{Q}$

$Q(\mathbf{W})_{\text{INT3}}$

+1	+0	−2	−3
−3	−4	+2	+1
−1	+2	−3	−2
−4	+2	+1	+0
+2	−2	−3	−3
+2	−4	−3	−4
+0	−4	+2	+3
+1	+3	−2	−2

(a) RTN quantization (**PPL 43.2**)

Perplexity:
A measure of how bad a model is in predicting the next token.
Lower is better

Naive 3 bit quantization increases perplexity a lot!

Not all Weights Matter the Same

$\mathbf{W}_{\text{FP16}}$

+1.2	−0.2	−2.4	−3.4
−2.5	−3.5	+1.9	+1.4
−0.9	+1.6	−2.5	−1.9
−3.5	+1.5	+0.5	−0.1
+1.8	−1.6	−3.2	−3.4
+2.4	−3.5	−2.8	−3.9
+0.1	−3.8	+2.4	+3.4
+0.9	+3.3	−1.9	−2.3

$\xrightarrow{Q}$

$Q(\mathbf{W})_{\text{MixPrec}}$

+1	+0	−2	−3
−2.5	−3.5	+1.9	+1.4
−1	+2	−3	−2
−4	+2	+1	+0
+2	−2	−3	−3
+2	−4	−3	−4
+0	−4	+2	+3
+1	+3	−2	−2

FP16 channel

(b) Keep 1% salient weights in FP16 (**PPL 13.0**)

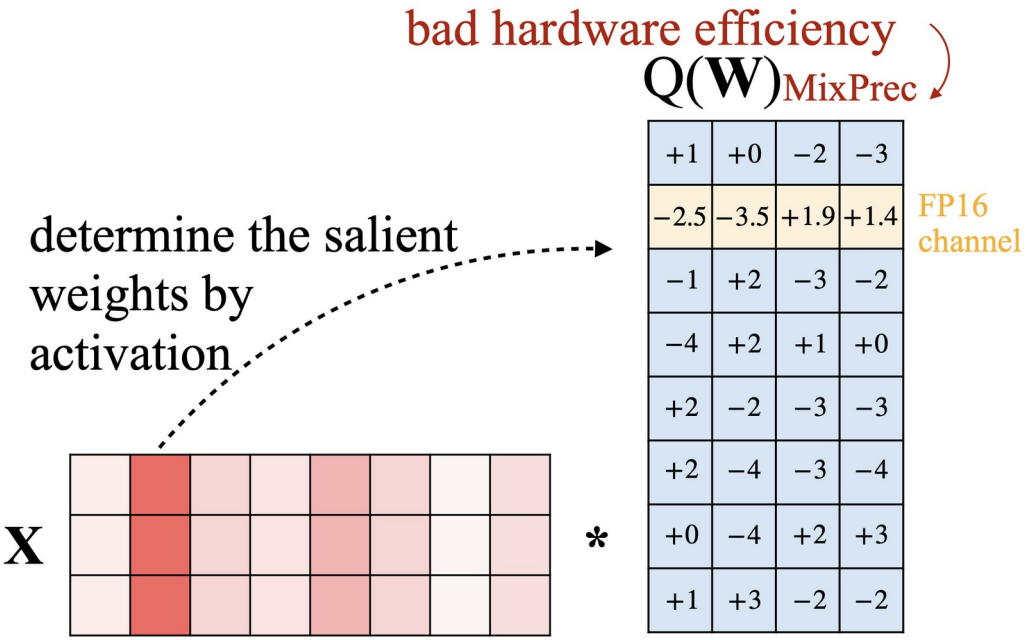
Two major questions to answer?

1- How to identify the salient weight channels?

2- How to get rid of a mixed precision kernel (which is difficult to implement)?

Keeping only 1% of salient weights in FP16 improves perplexity

How to Identify Salient Weights?



Recall the insight from SmoothQuant

- Some activations are quite large and we want to preserve them

Keeping salient channels in FP16 is challenging

- Mixed precision kernel is difficult to implement efficiently

Use large activations to determine which weights are salient

AWQ

$Q(\mathbf{W})$

+1	+0	−2	−3	x 1
−2.5	−3.5	+1.9	+1.4	x s
−1	+2	−3	−2	x 1
−4	+2	+1	+0	x 1
+2	−2	−3	−3	x 1
+2	−4	−3	−4	x 1
+0	−4	+2	+3	x 1
+1	+3	−2	−2	x 1

$$WX \longrightarrow Q(W.s)(s^{-1}.X)$$

Fused to the previous operation (e.g. a layernorm)

Authors find that s>0 will reduce perplexity (exact values determined empirically)

Quantization

- Quantization
  - What is it?
  - Why does it work?
- Linear Quantization
- Non-Linear Quantization
- Quantization-Aware Training
- LLMs and Quantization